

CSA1717-ARTIFICIAL INTELLIGENCE

PRACTICAL LAB EXPERIMENTS

RAGAVI S – (192565027)

EXPERIMENT 1: 8-PUZZLE PROBLEM

Aim

The aim of this experiment is to solve the 8-Puzzle Problem using the A* Search algorithm by moving tiles on a 3×3 grid from a given starting configuration to a target goal state.

Objective

To implement the A* search algorithm and the Manhattan Distance heuristic function to model a 3×3 grid and find the shortest path to solve the puzzle.

Algorithm

Step 1: Define the target goal state of the puzzle grid as (1, 2, 3, 4, 5, 6, 7, 8, 0).

Step 2: Accept the initial starting configuration of the puzzle from the user.

Step 3: Compute the initial Manhattan Distance (h) by calculating how many steps each misplaced number sits away from its correct goal position.

Step 4: Place the starting configuration into a priority queue, sorted by the total cost ($f = g + h$), where g is the number of moves made so far.

Step 5: Retrieve the node with the lowest total cost (f) from the priority queue.

Step 6: Check if this configuration matches the goal state. If yes, print the step-by-step path to the solution and stop.

Step 7: If it is not the goal, locate the empty tile (0) and swap it with its adjacent numbers (Up, Down, Left, or Right) to generate new child board states.

Step 8: For each new child state, check if it has been visited before. If it is unvisited, calculate its cost (f), add it to the queue, and return to Step 5.

Source Code

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def h(state):
```

```

    return sum(abs(i//3 - GOAL.index(x)//3) + abs(i%3 - GOAL.index(x)%3)
               for i, x in enumerate(state) if x)

def get_neighbors(state):
    neighbors = []

    blank = state.index(0)

    r, c = divmod(blank, 3)

    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
        if 0 <= r+dr < 3 and 0 <= c+dc < 3:
            target = (r+dr)*3 + (c+dc)

            new_state = list(state)

            new_state[blank], new_state[target] = new_state[target], new_state[blank]

            neighbors.append(tuple(new_state))

    return neighbors

def solve(start):
    queue = [(h(start), 0, start, [])]

    visited = {start}

    while queue:
        _, g, curr, path = heapq.heappop(queue)

        if curr == GOAL: return path + [curr]

        for nxt in get_neighbors(curr):
            if nxt not in visited:
                visited.add(nxt)

                heapq.heappush(queue, (g + 1 + h(nxt), g + 1, nxt, path + [curr]))

    return None

# --- Custom Input Handling ---

if __name__ == "__main__":
    user_in = input("Enter 9 numbers (0-8) separated by spaces: ")

    start_state = tuple(map(int, user_in.split()))

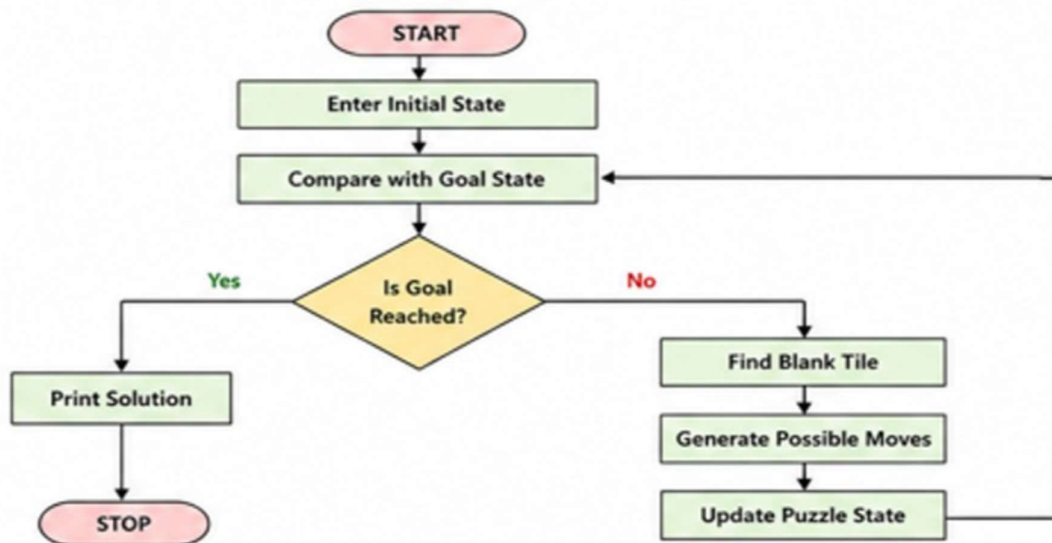
```

```

if len(start_state) != 9 or set(start_state) != set(range(9)):
    print("Error: Please enter exactly 9 unique digits from 0 to 8.")
else:
    solution = solve(start_state)
    if solution:
        print(f"\nSolved in {len(solution)-1} steps:")
        for step, state in enumerate(solution):
            print(f"Step {step}:")
            for i in range(0, 9, 3): print(list(state[i:i+3]))
            print()
    else:
        print("This specific puzzle layout is mathematically unsolvable!")

```

Flowchart



Output

```
>>>==== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/EXP1.py ====
Enter 9 numbers (0-8) separated by spaces: 1 2 3 4 8 0 7 6 5

Solved in 5 steps:
Step 0:
[1, 2, 3]
[4, 8, 0]
[7, 6, 5]

Step 1:
[1, 2, 3]
[4, 8, 5]
[7, 6, 0]

Step 2:
[1, 2, 3]
[4, 8, 5]
[7, 0, 6]

Step 3:
[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

Step 4:
[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

Step 5:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

>>>
```

Result

The 8-Puzzle problem was successfully solved using the A* Search algorithm by finding the fastest sequence of tile moves from the initial state to the target goal state.